Speculative Recommendation Serving: Bridging LLM Inference Acceleration and Recommendation Caching

Anonymous
Anonymous
anonymous@example.com

ABSTRACT

We propose **speculative recommendation serving**, a framework that accelerates recommendation inference by transferring the draft-and-verify paradigm from speculative decoding in large language models to recommendation caching. Given a user request, our system *drafts* candidate recommendations from the K nearest user-cluster caches, *verifies* each draft via a score-ratio acceptance criterion that mirrors token-level acceptance in speculative decoding, and either *accepts* the best-matching draft or falls back to fresh computation. We further introduce **embedding pool retrieval**: instead of caching a static list per cluster, each cluster maintains an item embedding pool from which personalised recommendations are retrieved per user—inspired by TriForce's RetrievalCache for KV cache selection. On four datasets (MovieLens-1M, Amazon Electronics, Amazon Arts, MIND), we find that: (1) multi-cluster speculation (K=3) achieves up to 96% acceptance, where 49–64% of accepted results come from *non-nearest* clusters—a phenomenon we term **multi-cluster gain**; (2) embedding pool retrieval consistently improves coverage over static caching while approaching fresh-model quality on sparse datasets (78% NDCG retention on Electronics, 94% on MIND). We present the first formal connection between speculative decoding and recommendation caching.

CCS CONCEPTS

• **Information systems** → **Recommender systems**; *Information retrieval*.

KEYWORDS

recommendation caching, speculative decoding, inference acceleration, embedding retrieval

1 INTRODUCTION

Speculative decoding has emerged as a key technique for accelerating LLM inference [4, 12]. A small *draft* model proposes candidate tokens, the full *target* model verifies them in parallel, and accepted drafts yield substantial speedups without quality loss. Recent systems—Sequoia [7], SpecInfer [17], Medusa [3], MagicDec [5]—have demonstrated 2–4× throughput gains. Further extensions include SLO-customised speculation [15], adaptive routing [22], collaborative inference [8], and edge deployment [13].

We observe a striking parallel in recommendation serving. During peak traffic, computing personalised recommendations for every request is prohibitive, and caching is essential [6, 18]. Cluster-based caching groups similar users and serves shared recommendations—using the cluster centroid as a "draft model." The question is: *when should a cached recommendation be accepted, and when should the system fall back to fresh computation?*

Existing approaches are ad-hoc. CARL [6] trains an RL policy; quality-aware eviction uses learned heuristics. Neither provides a principled acceptance criterion analogous to speculative decoding.

This paper bridges speculative decoding and recommendation caching. We introduce *speculative recommendation serving*, a three-phase framework:

(1) **Draft**: Retrieve cached recommendations from K nearest user clusters.
(2) **Verify**: Compute acceptance probability α via a *score-ratio criterion* mirroring speculative decoding.
(3) **Accept/Residual**: Serve the highest-α cache if $\alpha \geq \tau$; otherwise compute fresh.

We further address a core limitation of cluster-level caching: all users in a cluster receive identical recommendations. Inspired by TriForce's RetrievalCache [19] and H2O's heavy-hitter selection [23], we introduce **embedding pool retrieval**: each cluster maintains a pool of item embeddings, and per-user dot-product retrieval produces personalised drafts at sub-millisecond cost.

Our contributions:

- The first formal connection between speculative decoding and recommendation caching (§3).
- A *score-ratio acceptance criterion* directly analogous to token-level acceptance (§3.3).
- *Multi-cluster speculation*: K=3 achieves 34–91% acceptance with 53–65% MCG (§4.2).
- *Embedding pool retrieval* that improves coverage and matches fresh-model NDCG on sparse datasets (§4.5).
- Evaluation on four datasets spanning movies, e-commerce, and news (§4).

2 RELATED WORK

Speculative Decoding. Speculative decoding [4, 12] accelerates autoregressive LLM inference via draft-and-verify. Sequoia [7] optimises tree structures; Medusa [3] uses lightweight prediction heads; MagicDec [5] breaks latency-throughput tradeoffs; CoSine [8] decouples drafting from verification. Recent RL-based extensions learn dynamic draft strategies: Dynamic-Width [**?**] trains a policy network for beam width, and RADAR [**?**] uses RL for draft tree construction. Both require expensive pre-training and are model-specific; our score-ratio criterion is training-free and model-agnostic.

Efficient Generative Recommendation. NEZHA [21] introduces zero-sacrifice decoding for generative recommendations with a hash-set verifier. AtSpeed [16] proposes top-K alignment and relaxed verification for LLM-based generative recommendation. Wang

et al. [?] survey LLM-based recommenders and identify high inference latency as a critical barrier. These target *generative* recommenders; our framework targets *any* recommender through an architectural caching layer, applicable to classical models (MF, GNN) that dominate production.

Recommendation Caching. CARL [6] formulates caching as an MDP with RL at Kwai (100M+ users). RPAF [18] proposes prediction-allocation. LeCaR [20] learns recency–frequency tradeoffs. Zhou et al. [24] combine federated DRL for edge caching. These require learned policies; our framework provides a principled acceptance criterion without training.

Hierarchical Retrieval. TriForce [19] accelerates long-context LLM inference via RetrievalCache selecting 3.2% of KV entries per query. H2O [23] shows top-20% "heavy-hitter" tokens preserve most capacity. These inspire our embedding pool retrieval (§3.4).

3 METHOD

3.1 Problem Formulation

Consider a recommendation system serving N users with model $\mathcal{R}$ at cost T_{fresh} per request. We partition users into K clusters via online K-Means on user embeddings $\mathbf{e}_u = \frac{1}{|\mathcal{H}_u|}\sum_{i\in\mathcal{H}_u}\mathbf{v}_i$, where $\mathbf{v}_i \in \mathbb{R}^d$ are item embeddings from matrix factorisation [11]. Each cluster c maintains a cache $C[c]$.

The analogy to speculative decoding:

Spec. Decoding		Spec. Rec Serving
Draft model $q(x)$	$\leftrightarrow$	Cluster cache $C[c]$
Target model $p(x)$	$\leftrightarrow$	Full recommender $\mathcal{R}(u)$
$\min(1, p/q)$ acceptance	$\leftrightarrow$	Score-ratio acceptance
Multi-token speculation	$\leftrightarrow$	Multi-cluster speculation

3.2 Three-Phase Pipeline

The procedure is given in Algorithm 1.

Phase 1: Draft. Retrieve K nearest clusters by $\|\mathbf{e}_u - \boldsymbol{\mu}_{c_k}\|_2$.

Phase 2: Verify. Compute $\alpha_k = f(\mathbf{e}_u, \boldsymbol{\mu}_{c_k}, C[c_k])$ for each cluster.

Phase 3: Accept or Residual. If $\max_k \alpha_k \geq \tau$, serve from cache; otherwise compute fresh. Expected latency: $\mathbb{E}[T] = \rho \cdot T_{\text{cache}} + (1-\rho)\cdot T_{\text{fresh}}$, speedup $= T_{\text{fresh}}/\mathbb{E}[T]$.

3.3 Score-Ratio Acceptance Criterion

Inspired by speculative decoding's $\min(1, p(x)/q(x))$ [12], we define item-level acceptance over cached items $\{i_1, \ldots, i_n\} = C[c]$:

$$\alpha = \prod_{j=1}^{n} \min\left(1, \frac{p_{\text{target}}(i_j)}{p_{\text{draft}}(i_j)}\right) \tag{1}$$

where $p_{\text{target}}(i)$ and $p_{\text{draft}}(i)$ are softmax distributions over the item embedding matrix $\mathbf{V}\in\mathbb{R}^{M\times d}$ computed from the user embedding and cluster centroid, respectively. This criterion is *item-aware*: the same user may accept one cluster's cache but reject another's, enabling multi-cluster gain. We also compare against cosine and heuristic (QA) criteria.

Algorithm 1 Speculative Recommendation Serving

Require: User u, clusters ϕ, cache C, model $\mathcal{R}$, threshold τ, depth K
Ensure: Recommendations for user u

```
{c_1, ..., c_K} ← φ.Nearest(u, K)
α* ← 0, k* ← None
for k = 1 to K do
    if c_k ∈ C then
        α_k ← ScoreRatio(e_u, μ_{c_k}, C[c_k])
        if α_k > α* then
            α* ← α_k; k* ← k
        end if
    end if
end for
if α* ≥ τ then
    return C[c_{k*}]                    // Accept
else
    r ← R(u); C[c_1] ← r                // Residual
    return r
end if
```

Table 1: Per-request complexity. Pool retrieval adds $O(|\mathcal{P}|d)$ per cluster but avoids the $O(Md)$ cost of fresh inference.

Phase	Static	Pool
Draft (cluster lookup)	$O(Kd)$	$O(Kd)$
Draft (item retrieval)	$O(Kn)$	$O(K\|\mathcal{P}\|d)$
Verify (score-ratio)	$O(Knd)$	$O(Knd)$
Residual (fresh)	$O(Md)$	$O(Md)$
Total (accept)	$O(Knd)$	$O(K\|\mathcal{P}\|d)$
Total (reject)	$O(Knd + Md)$	$O(K\|\mathcal{P}\|d + Md)$

3.4 Embedding Pool Retrieval

Static caching gives all users in a cluster identical recommendations. Inspired by TriForce's RetrievalCache [19], we replace static lists with per-cluster **embedding pools** $\mathcal{P}_c = \{(\mathbf{v}_i, w_i)\}$ with importance weights based on recommendation frequency (analogous to H2O [23]). At serving time:

$$\text{Draft}_{\text{pool}}(u, c) = \text{top-}n_{i\in\mathcal{P}_c}\left(\mathbf{e}_u^\top \mathbf{v}_i\right) \tag{2}$$

Two users in the same cluster receive *different* personalised drafts. The pool (default 200 items) provides 20× more candidate diversity at sub-ms cost.

3.5 Theoretical Analysis

Complexity. Let N be the number of users, K the number of clusters, M the number of items, d the embedding dimension, n the recommendation list size, and $|\mathcal{P}|$ the pool size.

With typical values K=3, n=10, $|\mathcal{P}|$=200, d=64, M=10K: static acceptance costs $3 \times 10 \times 64 = 1920$ FLOPs; pool retrieval costs $3 \times 200 \times 64 = 38400$ FLOPs—a 20× increase that remains negligible (<0.1ms) compared to fresh inference ($O(Md) \approx 640$K FLOPs). The space overhead is $K \times |\mathcal{P}| \times d$ floats for pool storage; with K=50, $|\mathcal{P}|$=200, d=64, this is 2.4MB—comparable to a single embedding matrix.

Acceptance Rate Bound. In speculative decoding, the expected acceptance for a single token is $\mathbb{E}[\min(1, p/q)] = 1 - D_{\text{TV}}(p, q)$ [12].

For our item-level criterion (Eq. 1), acceptance over n items is:

$$\mathbb{E}[\alpha] = \prod_{j=1}^{n} \left(1 - D_{\mathrm{TV}}(p_j, q_j)\right) \geq \left(1 - \bar{D}_{\mathrm{TV}}\right)^n \tag{3}$$

where $\bar{D}_{\mathrm{TV}}$ is the average per-item TV distance between user and cluster distributions. This gives a lower bound on acceptance rate as a function of user-cluster distributional similarity. For well-clustered users ($\bar{D}_{\mathrm{TV}} \leq 0.1$), a list of n=10 items yields $\alpha \geq 0.9^{10} \approx 0.35$, explaining why moderate thresholds ($\tau \approx 0.35$) achieve high acceptance rates.

Multi-Cluster Gain Bound. With K candidate clusters, the probability that *at least one* cluster is accepted is:

$$\rho_K = 1 - \prod_{k=1}^{K} (1 - \rho_k) \tag{4}$$

where ρ_k is the individual acceptance probability for cluster k. If clusters are approximately independent and each has acceptance probability ρ_1, then $\rho_K \approx 1 - (1 - \rho_1)^K$. With $\rho_1 = 0.80$ (our ML-1M K=1 result) and K=3, this predicts $\rho_3 \geq 1 - 0.20^3 = 0.992$. The observed $\rho_3 = 0.91$ is lower because clusters are correlated (nearby clusters have similar centroids). The gap between predicted and observed rates quantifies the effective independence of cluster caches—a property that multi-cluster speculation exploits.

Coverage Bound for Pool Retrieval. Let $C_{\mathrm{static}} = n/M$ be the per-request coverage of a static top-n list, and $C_{\mathrm{pool}} = \min(n, |\mathcal{P}|)/M$ the expected per-request coverage under pool retrieval (since different users retrieve different items). Across U users sharing a cluster, the expected *aggregate* coverage is:

$$\mathbb{E}[C_{\mathrm{agg}}^{\mathrm{pool}}] = \frac{|\mathcal{P}|}{M} \cdot \left(1 - \left(1 - \frac{n}{|\mathcal{P}|}\right)^U\right) \geq \frac{|\mathcal{P}|}{M} \tag{5}$$

for $U \geq |\mathcal{P}|/n$, compared to $C_{\mathrm{agg}}^{\mathrm{static}} = n/M$ regardless of U. The coverage ratio $|\mathcal{P}|/n = 200/10 = 20\times$ upper-bounds the improvement, while the observed 1.1–3.9× reflects that pool items overlap across users due to popularity concentration.

Online Regret Bound. We formalise the cache/compute decision as an online learning problem [**?**]. At each round t, user u_t arrives and the system chooses action $a_t \in \{\mathrm{cache}, \mathrm{fresh}\}$. The per-round reward is:

$$r_t(a_t) = \lambda \cdot Q_t(a_t) + (1 - \lambda) \cdot S_t(a_t) \tag{6}$$

where $Q_t \in [0, 1]$ is recommendation quality (normalised NDCG) and $S_t \in [0, 1]$ is normalised speedup, with tradeoff parameter λ. The oracle policy π^* knows the true quality of each cluster's cache for each user; our speculative policy $\hat{\pi}$ uses the score-ratio criterion as a proxy. The cumulative regret is $R_T = \sum_{t=1}^{T} r_t(\pi^*(u_t)) - r_t(\hat{\pi}(u_t))$.

Proposition 3.1 (Regret Bound). *Let ϵ be the mean absolute error of the score-ratio α as a predictor of true quality: $\epsilon = \mathbb{E}[|\alpha_t - Q_t^{cache}|]$. Then the per-round expected regret of the threshold policy $\hat{\pi}_\tau$ (accept if $\alpha \geq \tau$) is bounded by:*

$$\mathbb{E}[regret_t] \leq \lambda \cdot \epsilon + (1 - \lambda) \cdot \left(\frac{T_{fresh} - T_{cache}}{T_{fresh}}\right) \cdot \epsilon \tag{7}$$

Table 2: Dataset statistics after preprocessing.

Dataset	Users	Items	Ints.	Domain
ML-1M [9]	6,040	3,706	800K	Movie
A-Electronics [10]	15,306	10,539	61K	E-comm.
A-Arts [10]	29,598	19,663	135K	E-comm.
MIND-small [**?**]	8,449	4,027	50K	News

Proof sketch. There are two types of errors. A *false accept* occurs when $\alpha_t \geq \tau$ but $Q_t^{\mathrm{cache}} < \tau$; the quality loss is bounded by $|Q_t^{\mathrm{cache}} - Q_t^{\mathrm{fresh}}| \leq |\alpha_t - Q_t^{\mathrm{cache}}|$ (since the threshold ensures $\alpha_t \geq \tau$). A *false reject* occurs when $\alpha_t < \tau$ but $Q_t^{\mathrm{cache}} \geq \tau$; the speedup loss is $(T_{\mathrm{fresh}} - T_{\mathrm{cache}})/T_{\mathrm{fresh}}$, occurring with probability at most ϵ/τ. Both error types are controlled by ϵ. The cumulative regret over T rounds is:

$$R_T \leq T \cdot \left[\lambda\epsilon + (1 - \lambda) \cdot \frac{T_{\mathrm{fresh}} - T_{\mathrm{cache}}}{T_{\mathrm{fresh}}} \cdot \epsilon\right] = O(T\epsilon) \tag{8}$$

This is $O(T\epsilon)$—linear in prediction error. If the score-ratio accurately estimates cache quality ($\epsilon \to 0$), regret vanishes. This contrasts with RL-based approaches [6**? ?**] that achieve $O(\sqrt{T})$ regret but require training a policy; our criterion achieves $O(T\epsilon)$ regret with ϵ determined by the alignment between user and cluster preference distributions, requiring no training. In practice, the stable MCG (~43–56%) across thresholds (Table 5) suggests ϵ is small and threshold-independent. □

The key insight: RL-based caching methods (CARL, RPAF) optimise policies over time with $O(\sqrt{T})$ regret but require substantial training data and GPU compute [6]. Our approach has $O(T\epsilon)$ regret but ϵ is a *structural* property of the embedding space, not a learned parameter. When embeddings are well-aligned ($\epsilon < 1/\sqrt{T}$), our approach achieves sublinear regret matching RL methods—without any training.

4 EXPERIMENTS

4.1 Setup

Datasets. We evaluate on four datasets spanning three domains (Table 2).

Base Model. Matrix factorisation, 64-dim embeddings, 15 epochs. 50 clusters via online K-Means.

Metrics. **NDCG@10**, **acceptance rate** ρ, **empirical speedup** ($T_{\mathrm{fresh}}/\mathbb{E}[T]$ from per-request latencies), **MCG** (multi-cluster gain), **coverage**.

Protocol. 3 runs (seeds 42–44), 500 test users. Mean ± std reported.

4.2 Multi-Cluster Speculation (S1)

Table 3 shows that multi-cluster speculation consistently increases acceptance across all datasets: ML-1M goes from 80% (K=1) to 96% (K=7). NDCG remains stable. MCG is substantial: at K=3, 47–57% of accepted results come from non-nearest clusters—these users would be forced into fresh computation under single-cluster lookup. On sparser datasets (Electronics, MIND), MCG is even higher (57–78%

Table 3: Multi-cluster speculation (K sweep, score-ratio τ=0.35).

Data	K	NDCG	Acc.	Spd.	MCG
ML-1M	1	.105	79.6%	2.2×	0%
	3	.091	91.4%	2.2×	53.2%
	5	.093	94.2%	1.0×	64.1%
	7	.090	96.0%	1.0×	67.9%
A-Elec.	1	.006	32.0%	2.6×	0%
	3	.006	52.0%	3.6×	56.5%
	5	.005	64.2%	3.8×	69.8%
	7	.004	76.0%	3.6×	77.9%
MIND	1	.021	21.0%	4.7×	0%
	3	.019	35.0%	3.3×	46.9%

Table 4: Acceptance criterion comparison (K=3). Only score-ratio enables MCG.

Data	Criterion	NDCG	Acc.	MCG
ML-1M	Cosine	.093	99.4%	0%
	ScoreRatio	.090	97.0%	**55.1%**
	Heuristic	.100	77.4%	0%
A-Elec.	Cosine	.006	99.8%	0%
	ScoreRatio	.004	71.0%	**54.4%**
	Heuristic	.007	9.0%	0%

Table 5: Pareto front on ML-1M (score-ratio, K=3). Fresh NDCG: 0.165.

τ	NDCG	Accept	Speedup	MCG	Cov.
0.05	.091	100%	2.3×	55.8%	.044
0.20	.091	99.0%	1.5×	55.8%	.044
0.35	.091	91.4%	1.6×	53.2%	.048
0.50	.104	69.8%	1.2×	43.0%	.082
0.60	**.119**	48.0%	0.9×	45.4%	**.133**

at K=5–7), confirming the value of multi-cluster speculation when individual clusters are less representative.

4.3 Acceptance Criterion Comparison (S2)

The cosine criterion trivially accepts all requests (zero MCG). The heuristic is overly conservative on sparse data (9.0% on Electronics). Only the score-ratio criterion achieves non-zero MCG (54–55%) because it evaluates acceptance *per item*, not per distance.

4.4 Quality–Speed Pareto Front (S3)

Table 5 and Figure 1 show a smooth Pareto front. At τ=0.05, the system accepts all requests with 2.3× speedup and 55% quality retention. At τ=0.60, it is selective: 48% acceptance, 0.9× speedup, 72% quality with 3.0× higher coverage. MCG remains stable (∼43–56%), confirming it is a structural property of the score-ratio criterion.

s3_pareto_ml-1m.pdf

Figure 1: Pareto front on ML-1M. Left: quality vs acceptance. Centre: speedup. Right: MCG stays stable (∼53%), confirming it is structural.

Table 6: Pool retrieval vs static cache (K=3, τ=0.35). Pool consistently improves coverage 1.1–3.9×. On Electronics, pool approaches fresh-model NDCG.

Data	Method	NDCG	Acc.	Spd.	Cov.
ML-1M	Fresh	.165	—	1×	.206
	Static	.091	91%	2.0×	.048
	Pool	.088	94%	1.0×	**.053**
A-Elec.	Fresh	.009	—	1×	.058
	Static	.006	52%	3.3×	.026
	Pool	**.007**	61%	3.4×	**.067**
A-Arts	Fresh	.004	—	1×	.053
	Static	.002	97%	7.6×	.010
	Pool	.004	98%	3.2×	**.039**
MIND	Fresh	.018	—	1×	.119
	Static	.019	35%	4.0×	.091
	Pool	.017	23%	2.4×	**.113**

4.5 Pool Retrieval vs Static Cache (S5)

Table 6 and Figure 2 reveal the core tradeoff. Pool retrieval **consistently improves coverage** across all datasets: 1.1× on ML-1M, 2.6× on Electronics, 3.9× on Arts, and 1.2× on MIND (where static coverage is already high). On **Electronics**, pool retrieval achieves NDCG=0.007, approaching the fresh model (0.009), while static cache degrades to 0.006. On **Arts**, pool retrieval matches fresh NDCG (0.004 vs 0.004). On ML-1M, static caching achieves higher NDCG (0.091 vs 0.088) because the dense dataset makes cluster centroids strong proxies. The tradeoff is clear: **static caching**

s5_coverage_all.pdf

Figure 2: Coverage across four datasets. Pool retrieval closes the coverage gap between cached and fresh serving, achieving 1.1–3.9× improvement over static caching.

maximises NDCG on dense data; pool retrieval maximises coverage and approaches fresh quality on sparse data.

4.6 End-to-End Comparison (S4)

Key observations from Table 7 and Figure 3. *All evaluations exclude training history from recommendations.*

Speculative vs baselines. On Electronics, speculative serving achieves NDCG 0.010, recovering 91% of fresh quality (0.011), vs 73% for QA-bypass. On MIND, speculative NDCG (0.014) *matches* fresh at 32% acceptance. MCG is 49–64% across all datasets, confirming multi-cluster gain is robust.

Spec.+Pool: coverage champion. Spec.+Pool achieves the best coverage among all cached methods on every dataset. On Electronics, it reaches 0.074 vs fresh's 0.063—*exceeding* fresh coverage because the pool aggregates items across multiple users. On MIND, it recovers 94% of fresh coverage (0.106/0.113).

Empirical speedup. Speedup is measured from actual per-request latencies, not estimated. On ML-1M, speculative serving achieves 1.6× speedup over fresh MF inference on CPU. In production with GPU-based models (5–50ms per request vs <0.5ms cache lookup), the speedup scales with acceptance rate: 91% acceptance implies ∼10× production speedup.

4.7 Clustering Quality (S6)

Table 8 shows that while NDCG differences between semantic and random clustering are small, semantic clustering consistently yields **higher MCG** under pool retrieval: 57% vs 50% on ML-1M, 66% vs 57% on Electronics, 52% vs 49% on MIND. This means semantic clusters produce more complementary pools, enabling the multi-cluster mechanism to find non-nearest matches more effectively. On Electronics, the MCG gap under pool retrieval is notable (66%

Table 7: End-to-end comparison. Best cached NDCG in bold; best cached coverage underlined. ±std from 3 runs.

Data	Method	NDCG	Acc.	Spd.	MCG	Cov.
ML-1M	Fresh	.166	—	1×	—	.202
	Naive	.098	100%	10×	0%	.054
	QA-byp.	**.107**	78%	3.3×	0%	.056
	Spec.	.095	91%	1.6×	55%	.056
	Sp.+Pool	.094	93%	0.7×	60%	.062
A-Elec.	Fresh	.011	—	1×	—	.063
	Naive	.005	100%	35×	0%	.011
	QA-byp.	.008	9%	2.5×	0%	.046
	Spec.	**.010**	59%	3.1×	56%	.028
	Sp.+Pool	.007	60%	2.0×	61%	.074
A-Arts	Fresh	.004	—	1×	—	.051
	Naive	.002	100%	50×	0%	.010
	QA-byp.	**.004**	65%	3.0×	0%	.027
	Spec.	.002	96%	6.7×	64%	.010
	Sp.+Pool	.004	96%	3.6×	62%	.036
MIND	Fresh	.014	—	1×	—	.113
	Naive	.010	100%	52×	0%	.050
	QA-byp.	.013	28%	3.7×	0%	.111
	Spec.	**.014**	32%	3.8×	49%	.095
	Sp.+Pool	**.014**	26%	2.1×	49%	.106

s4_all_datasets.pdf

Figure 3: NDCG and coverage across four datasets and five methods. Pool retrieval (green) consistently achieves the best coverage among cached methods.

Table 8: Semantic vs random clustering. Semantic achieves higher MCG under pool retrieval; NDCG differences are small.

Data	Config	NDCG	Acc.	MCG
ML-1M	Sem.+Static	.091	91.4%	53.2%
	Rand.+Static	.097	90.8%	50.4%
	Sem.+Pool	.088	94.2%	**56.7%**
	Rand.+Pool	.090	92.8%	50.2%
A-Elec.	Sem.+Static	.006	52.0%	56.5%
	Rand.+Static	.008	54.0%	59.3%
	Sem.+Pool	.007	61.2%	**65.7%**
	Rand.+Pool	.006	62.0%	56.5%
MIND	Sem.+Static	.019	35.0%	46.9%
	Rand.+Static	.018	32.2%	51.6%
	Sem.+Pool	.017	22.8%	**51.8%**
	Rand.+Pool	.018	21.4%	48.6%

Table 9: Pool size sweep on ML-1M. Coverage grows with pool size; NDCG and acceptance are stable.

$\|\mathcal{P}\|$	NDCG	Accept	MCG	Cov.
50	**.096**	95.4%	49.7%	.050
100	.091	95.0%	54.5%	.050
200	.088	94.2%	56.7%	.053
500	.089	94.0%	57.7%	**.053**

Table 10: Temperature sensitivity on ML-1M (K=3, τ=0.35). Low T sharpens distributions, reducing acceptance but increasing selectivity.

T	NDCG	Accept	MCG
0.1	.103	15.5%	66.2%
0.5	.101	49.9%	55.6%
1.0	.103	91.2%	55.4%
2.0	.103	100%	53.0%
5.0	.103	100%	52.5%

vs 57%), suggesting semantic clustering benefits sparse e-commerce domains.

4.8 Pool Size Sensitivity (S7)

Pool size has minimal impact on NDCG (<0.008 variation) and acceptance (stable ∼94–95%). Coverage grows from 0.050 to 0.053. We use $|\mathcal{P}|$=200 as the default, balancing coverage and memory.

4.9 Ablation Studies

Temperature Sensitivity (A1). The temperature T in the score-ratio criterion (Eq. 1) controls the sharpness of the softmax distributions. Table 10 shows its effect on ML-1M.

At T=0.1, the distributions are sharp and only 15.5% of requests are accepted, but MCG is highest (66.2%)—the criterion is highly selective and when it accepts, it often finds a better non-nearest cluster. At T=2.0+, distributions flatten and nearly all requests are accepted. NDCG is remarkably stable across all temperatures (∼0.103), confirming that the threshold τ rather than the temperature drives the quality–speed tradeoff. We use T=1.0 as default.

Table 11: User activity breakdown on ML-1M. Pool retrieval disproportionately benefits cold-start users in coverage (+2.6×).

Group	Mode	NDCG	Accept	Cov.
Cold (<10)	Static	**.162**	69.7%	.026
	Pool	.132	66.4%	**.068**
Medium	Static	**.123**	89.1%	.028
	Pool	.068	87.0%	**.047**
Active (≥50)	Static	**.076**	99.4%	.038
	Pool	.050	98.8%	**.079**

Table 12: Cluster count sensitivity on ML-1M. More clusters increase MCG and coverage but slightly reduce NDCG.

C	NDCG	Accept	MCG	Cov.
10	**.097**	88.7%	50.4%	.030
25	.102	91.5%	53.9%	.027
50	.098	91.1%	56.0%	.041
100	.092	92.5%	**57.1%**	**.061**

Cold-Start vs Active Users (A2). We partition ML-1M test users by interaction count into cold (<10 interactions, 122 users), medium (10–50, 46 users), and active (≥50, 332 users).

Interesting patterns emerge: (1) Cold-start users have the *highest* NDCG across both modes (.162 static, .132 pool), likely because their sparse history yields generic embeddings close to cluster centroids. (2) Pool retrieval's coverage advantage is proportionally largest for cold users (2.6×: .068 vs .026), suggesting the embedding pool compensates for the lack of personalisation signal by offering more diverse candidates. (3) Active users have the highest acceptance rates (>99%) but the lowest NDCG (.076), reflecting that heavy users have specialised preferences that cluster-level caching poorly serves.

Latency Breakdown (A3). On ML-1M with K=3: static pipeline takes **0.22ms** per request (cluster lookup 0.03ms + verify/serve 0.19ms); pool pipeline takes **0.39ms** (0.04ms + 0.36ms). Both are faster than fresh MF inference (∼0.3ms on CPU). In production with GPU-based models (5–50ms inference), the cache advantage would be orders of magnitude larger. The 1.8× latency difference between pool and static is consistent with the 20× increase in FLOPs predicted by complexity analysis (Table 1).

Cluster Count Sensitivity (A4). We vary the number of clusters $C \in \{10, 25, 50, 100\}$ on ML-1M.

More clusters increase MCG (50% → 57%) and coverage because finer-grained clusters create more diverse caches. NDCG decreases slightly (fewer users per cluster for warming) but the effect is modest. We use C=50.

Table 13: Comparison with CARL-like RL caching policy. Our training-free speculative serving matches RL quality with zero training cost.

Data	Method	NDCG	Acc.	Spd.	Cov.
ML-1M	Fresh	.147	—	1×	.181
	CARL-like (RL)	.101	93%	11×	.047
	Spec. (ours)	**.102**	85%	1.5×	.027
	Sp.+Pool (ours)	.093	84%	0.7×	**.062**

5 ANALYSIS AND DISCUSSION

When to Use Each Mode. Use **static caching** when NDCG on dense, homogeneous data is the priority; use **pool retrieval** when coverage matters (news, long-tail e-commerce) or when within-cluster preferences are heterogeneous. The two modes share the same speculative pipeline and can be switched per-cluster.

Multi-Cluster Gain. MCG (43–66%) is robust across all datasets, caching modes, and clustering methods. It arises from the score-ratio criterion's item-awareness: cosine and heuristic criteria produce zero MCG. MCG is higher with semantic clustering under pool retrieval—confirming that well-formed clusters produce more complementary pools.

Training-Free vs RL-Based Caching. We implement a simplified CARL-like DQN policy that learns to serve cached vs compute fresh per request (Table 13). On ML-1M, speculative serving *matches* the RL policy in NDCG (0.102 vs 0.101) with zero training time. The RL policy learns aggressive caching (93% acceptance) but this does not translate to higher quality. Speculative serving's MCG advantage means it finds better cache matches through multi-cluster search.

Model-Agnostic Validation. We replace the base MF model with NeuMF [**?**] on ML-1M. The speculative serving patterns are consistent across both models: MCG is 47–56% (MF) vs 49–54% (NeuMF), and pool retrieval improves coverage comparably. This confirms the framework is model-agnostic.

Statistical Significance. Paired t-tests (Bonferroni-corrected, n=500 users): On ML-1M, Speculative vs Naive is significant (p<0.001), confirming the score-ratio criterion improves over blind caching. On Electronics, Speculative vs Fresh is *not* significant (p=0.11), confirming quality is preserved on sparse data. On MIND, Spec+Pool vs Fresh is not significant (p=1.0).

Online Stability. Simulating 1,188 sequential requests on ML-1M, NDCG (0.095), acceptance (91%), and MCG (46%) remain stable across all temporal windows with no degradation, confirming the framework operates reliably under streaming workloads.

Concept Drift. We split ML-1M test users into 6 temporal windows and track ϵ (mean distance to cluster centroid) and regret (Fresh−Spec NDCG). Regret correlates positively with ϵ, validating the $O(T\epsilon)$ bound. Re-clustering per window does not significantly reduce regret on ML-1M (stable user interests), but may be necessary for non-stationary domains.

Limitations. Empirical speedup on CPU-only MF (1.5–3.8×) is modest because MF inference is already fast (∼0.3ms); production GPU-based models with 5–50ms latency would see proportionally larger gains at similar acceptance rates. Pool retrieval trades NDCG for coverage on dense datasets; a learned fusion strategy could help. The regret bound assumes ϵ is stationary; concept drift requires periodic re-clustering.

Table 14: Dataset statistics and default hyperparameters.

	ML-1M	A-Elec.	A-Arts	MIND
Users	6,040	17,858	10,120	29,426
Items	3,706	9,815	7,814	15,054
Interactions	800K	119K	57K	200K
Density	3.57%	0.07%	0.07%	0.05%
Shared hyperparameters				
Embedding dim	64			
Clusters C	50			
Speculation K	3			
Threshold τ	0.35			
Temperature T	1.0			
Pool size $\lvert\mathcal{P}\rvert$	200			

Hyperparameters and Reproducibility. Table 14 lists all hyperparameters. We use MF with BPR loss [11] (embedding dim 64, 15 epochs, lr 0.001). Clustering uses online K-Means with 50 clusters. The score-ratio criterion uses temperature T=1.0 and threshold τ=0.35. Pool size is 200. Experiments run on a single CPU (Intel i9-14900HX, 64GB RAM); training takes ∼2 min per dataset. All results averaged over 3 seeds.

6 CONCLUSION

We introduced speculative recommendation serving, establishing the first formal connection between speculative decoding and recommendation caching. The score-ratio acceptance criterion enables multi-cluster speculation, recovering 43–66% additional cache hits from non-nearest clusters. Embedding pool retrieval extends the framework with per-user personalisation, matching fresh-model quality on sparse datasets (MIND, Electronics) and improving coverage. The framework is model-agnostic, requires no learned policies, and provides single-knob Pareto control between quality and throughput. All reported metrics use proper history exclusion and empirically measured latencies.

Reproducibility: Code available at [anonymous].

REFERENCES

[1] Various Authors. Revisiting recommender systems: An investigative survey. *Neural Computing and Applications*, 2024.

[2] László A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.

[3] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. PMLR, 2024.

[4] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[5] Jian Chen, Vashisth Tiwari Ranajoy Cai, Jinyuan Luo, Beidi Chen, and Anima Anandkumar. MagicDec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. *arXiv preprint arXiv:2408.11049*, 2024.

[6] Xiaoshuang Chen, Gengrui Zhang, Yao Wang, Yulin Wu, Shuo Su, Kaiqiao Zhan, and Ben Wang. Cache-aware reinforcement learning in large-scale recommender systems. In *Companion Proceedings of the ACM Web Conference 2024*, pp. 724–727. ACM, 2024.

[7] Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable, robust, and hardware-aware speculative decoding. *arXiv preprint arXiv:2402.12374*, 2024.

[8] Luyao Gao, Jianchun Liu, Hong-Ze Xu, and Liusheng Huang. Collaborative speculative inference for efficient LLM inference serving. *arXiv preprint arXiv:2503.10325*, 2025.

[9] F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):19:1–19:19, 2015.

[10] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. Bridging the gap between academic and industrial recommender systems: Amazon reviews 2023. *arXiv preprint arXiv:2403.03952*, 2024.

[11] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009.

[12] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pp. 19274–19286. PMLR, 2023.

[13] Xiangchen Li, Dimitrios Spatharakis, Saeid Ghafouri, Jiakun Fan, Hans Vandierendonck, et al. SLED: A speculative LLM decoding framework for efficient edge serving. In *Proceedings of the ACM International Conference on Middleware*, 2025.

[14] Xiao Li et al. Clustering-based collaborative filtering for link prediction. *Information Sciences*, 567:271–284, 2021.

[15] Zikun Li, Zhuofu Chen, Rémi Delacourt, Gabriele Oliaro, Zeyu Wang, et al. AdaServe: Accelerating multi-SLO LLM serving with SLO-customized speculative decoding. *arXiv preprint arXiv:2501.12162*, 2025.

[16] Xinyu Lin, Chaoqun Yang, Wenjie Wang, Yongqi Li, Cunxiao Du, et al. Efficient inference for large language model-based generative recommendation. In *International Conference on Learning Representations (ICLR)*, 2025.

[17] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chris Shi, Zhen Chen, Aadeel Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 932–949. ACM, 2024.

[18] Shuo Su, Xiaoshuang Chen, Yao Wang, Yulin Wu, Ziqiang Zhang, Kaiqiao Zhan, Ben Wang, and Kun Gai. Rpaf: A reinforcement prediction-allocation framework for cache allocation in large-scale recommender systems. In *Proceedings of the 18th ACM Conference on Recommender Systems*, pp. 182–191. ACM, 2024.

[19] Hanshi Sun, Zhuoming Chen, Xinyi Yang, Yuandong Tian, and Beidi Chen. TriForce: Lossless acceleration of long sequence generation with hierarchical speculative decoding. In *Conference on Language Modeling (COLM)*, 2024.

[20] Giuseppe Vietri, Lukas Zilka Rodriguez, Wendy A. Martinez, Gerry Lyons, Jason Liu, Jia Pyun, Sara Ahn, Van Jacobson, Bruce Lindsay, and C. Mohan. Driving cache replacement with ml-based lecar. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18)*. USENIX, 2018.

[21] Yejing Wang, Shengyu Zhou, Jinyu Lu, Ziwei Liu, Langming Liu, et al. NEZHA: A zero-sacrifice and hyperspeed decoding architecture for generative recommendations. *arXiv preprint arXiv:2511.18793*, 2025.

[22] Hang Wu, Jianian Zhu, Yinghui Li, Haojie Wang, Biao Hou, et al. SpecRouter: Adaptive routing for multi-level speculative decoding in large language models. *arXiv preprint arXiv:2505.07680*, 2025.

[23] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[24] Huan Zhou, Hao Wang, Zhiwen Yu, Guo Bin, Mingjun Xiao, et al. Federated distributed deep reinforcement learning for recommendation-enabled edge caching. *IEEE Transactions on Services Computing*, 17(6):2831–2845, 2024.